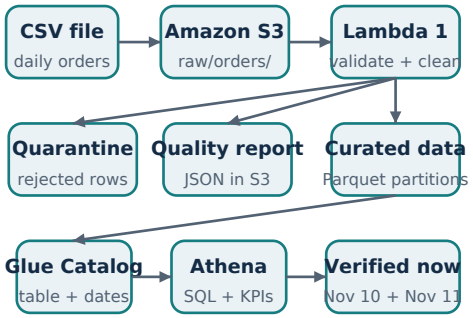
SERVERLESS AWS ETL PIPELINE

A simple, step-by-step guide to understand, build, test and explain the project

Think of it as a smart parcel-sorting centre. Every day, a shop sends a file of orders. Our system checks each order, separates good records from bad ones, stores the clean data, and lets us ask questions about sales.

PROJECT ARCHITECTURE - CURRENT BUILD VS. NEXT PHASES



NEXT PHASE - NOT YET IMPLEMENTED

SNS email alerts | Lambda 2 + Gemini | EventBridge daily schedule — Optional: SQS + DLQ

Prepared: 12 November 2026 | AWS Region: ap-southeast-2 (Sydney) | Project status: ingestion and analytics foundation verified; AI reporting and alerting remain to be built.

1. First, what is this document?

This document teaches the project from zero. It uses simple language first, then shows the real AWS services, code examples, commands and measured results. You can use it to learn the system and to prepare for an interview.

Important honesty note: this is a progress report of the implementation verified so far, not a claim that every planned feature is finished. The S3-to-Lambda ingestion pipeline, Parquet outputs, Glue Catalog and Athena queries have been tested. SNS email alerting, the second Lambda, Gemini integration and EventBridge scheduling are still pending.

Project status at a glance

Part	Status	What the evidence says
AWS account, CLI and S3	Complete	Account setup, CLI authentication, bucket, encryption and public-access block verified.
CSV generator and local Python setup	Complete	Python 3.14.6 environment and sample CSVs created.
Lambda 1 validation and cleaning	Complete	Manual invocation processed 5,050 rows and produced outputs.
Automatic S3 trigger	Complete	A new upload created its report about 7 seconds later; Parquet output also appeared.
Glue Catalog and Athena	Complete	Two date partitions queried successfully; KPI results obtained.
SNS email alerts	Pending	Topic, subscription confirmation and real email test not yet completed.
Lambda 2 + Gemini	Pending	Not yet implemented or tested.
EventBridge schedule	Pending	Not yet configured.
SQS + DLQ	Optional / pending	Reliability hardening; requires approval and architecture change.
Final README and report	In progress	This learning report is being created; repository packaging remains.

Status labels mean: Complete = evidence was observed; Pending = not built or not verified; Optional = an enhancement, not required for the core pipeline.

2. Explain the project like you are in first class

The parcel-sorting story

Imagine a toy shop. Every evening, the shop sends a big list of everything it sold. Some lines are correct. Some lines are broken: a customer number is missing, a quantity is zero, or the same order appears twice.

We build a helper system that does five simple jobs:

- Receive: take the shop's list and put it in a safe warehouse.
- Check: read every line and see whether it follows the rules.
- Sort: put correct lines in the clean-data box and incorrect lines in the reject box.
- Count: calculate how many orders, customers and items there were, and how much money the shop made.
- Explain: in a later phase, use AI to write a short, human-readable daily summary.

What does ETL mean?

Letter	Word	Simple meaning	In this project
E	Extract	Get the data from somewhere.	Read the CSV uploaded to S3.
T	Transform	Clean and change the data.	Validate rows and convert valid data to Parquet.
L	Load	Put the result where it can be used.	Save Parquet files in S3 and register them in Glue.

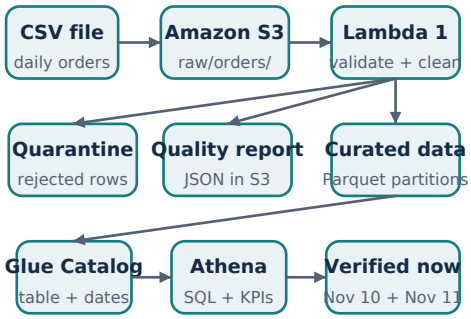
One-line definition: ETL is the process of taking raw data, making it reliable, and storing it in a form that people and tools can analyse.

Why serverless?

Serverless means we use managed cloud services instead of maintaining our own always-running server for this workload. S3 stores files, Lambda runs code when triggered, and Athena runs SQL queries when requested. It does not mean the project has zero cost; AWS usage can still be billed.

3. The whole system at one glance

PROJECT ARCHITECTURE - CURRENT BUILD VS. NEXT PHASES



NEXT PHASE - NOT YET IMPLEMENTED

SNS email alerts | Lambda 2 + Gemini | EventBridge daily schedule — Optional: SQS + DLQ

The solid upper and middle path is the part already built and tested. The amber dashed area is planned work. The AI report path is not yet a working feature.

The journey of one CSV file



What happens in order?

- A CSV file is generated locally and uploaded to the S3 folder `raw/orders/`.
- S3 notices the new `.csv` object and sends an event to the ingestion Lambda.
- Lambda reads the file, checks the columns and validates each row.
- Valid rows are written as Parquet files under a date partition.
- Rejected rows are written to a quarantine CSV; a JSON quality report is written to S3.
- Glue stores the table definition and partition information.
- Athena runs SQL over the Parquet files and returns daily KPIs.
- Later: Lambda 2 will use those KPIs, ask Gemini to explain them, save a report and send an email.

4. Meet the AWS services

Service	Kid-friendly comparison	Actual job here
Amazon S3	A warehouse with labelled shelves.	Stores raw CSV, clean Parquet, rejected CSV, reports and Athena results.
AWS Lambda	A worker who wakes up when a bell rings.	Runs Python code after an S3 event; no server is managed by us.
IAM	A key card with specific permissions.	Lets Lambda read or write only the resources it needs.
AWS Glue Data Catalog	A library catalogue.	Stores the table schema and partition locations so Athena can find the data.
Amazon Athena	A question desk for data.	Runs SQL queries directly over data in S3.
Amazon SNS	A phone tree / notification service.	Planned: sends a message when the rejection rate is too high and emails reports.
Amazon EventBridge	An alarm clock.	Planned: starts the daily insights Lambda at a chosen time.
Google Gemini API	A writing assistant.	Planned: turns verified KPI numbers into a short business explanation.
Amazon SQS + DLQ	An inbox and a failed-message box.	Optional: buffers events and keeps messages that fail repeatedly.

Why not use one service for everything?

Each service has one main responsibility. S3 stores files; Lambda runs code; Glue describes the data; Athena analyses it. Keeping responsibilities separate makes the system easier to test, troubleshoot and explain.

5. The data: what is inside an order?

The generator creates synthetic store-order data. Synthetic means the records are made for testing; they are not real customer transactions. The current CSV has six main columns:

Column	Meaning	Example / expected type
order_id	Unique identifier for an order.	ORD-... / text
customer_id	Identifier for the customer.	integer-like value
category	Product category.	text
quantity	Number of units in the order.	positive whole number
unit_price	Price per unit.	positive number
order_ts	Time when the order was placed.	date and time

What makes a row invalid?

- An order ID is missing.
- A customer ID is missing or is not a valid integer.
- Quantity is missing, zero, negative or not a whole number.
- Unit price is missing, zero or negative.
- Order timestamp is missing or cannot be parsed.
- Category is missing.
- The order ID is a duplicate within the same file (the first occurrence is kept).

Important limit: duplicate detection is currently within one input file. It does not prove that the same order ID will be detected across different days/files.

How the test generator makes defects

The generator can deliberately inject bad rows. The default defect rate used for the two verified daily files was 1% for each of four invalid-data groups, plus duplicate rows. This created 250 rejected rows out of 5,050 input rows. A higher defect rate can be used for a future alert test.

PowerShell examples - generator commands

```
python .\\scripts\\generate_orders.py 2026-11-10
python .\\scripts\\generate_orders.py 2026-11-11
# For a future high-defect alert test, after checking the date is unused:
python .\\scripts\\generate_orders.py 2026-11-12 --defect-rate 0.06
```

The generator's `--defect-rate` option controls how many examples of each injected defect group are added. The exact final rejection rate must always be measured from the output; it should not be assumed from the setting alone.

6. What we set up on the computer

Project folder

Current project structure (simplified)

```
ETL/
|-- .venv/
|-- data/
|-- docs/
|-- scripts/
|   |-- generate_orders.py
|-- sql/
|   |-- 01_validate_orders.sql
|   |-- 02_daily_kpis.sql
|-- src/
|   |-- lambda_ingest/
|   |   |-- lambda_function.py
|   |-- lambda_insights/
|-- lambda_ingest.zip
```

The `.venv` folder keeps this project's Python packages separate from other Python projects on the computer. `scripts` contains the CSV generator, `src` contains Lambda code, `sql` contains Athena queries, and `docs` holds test/configuration artefacts.

Python packages installed

Package	Why it is used
pandas	Read, inspect and transform tabular data.
NumPy	Generate numeric sample data.
awswrangler	Read/write data in AWS services, including S3 Parquet.
boto3	Python SDK for AWS services such as S3 and SNS.
PyArrow	Parquet support and columnar data handling.

AWS CLI authentication

The AWS CLI was installed and authenticated using browser-based login with the `etl-project` profile. We did not create permanent IAM access keys for this project.

Useful verification commands

```
aws sts get-caller-identity --profile etl-project
aws configure get region --profile etl-project
```

The configured region is `ap-southeast-2` (Sydney). Keeping one region consistent matters because Lambda, Glue, Athena and S3 integrations are region-aware.

7. Amazon S3: the project's file warehouse

We created one private S3 bucket:

Bucket name

```
orders-etl-pipeline-2026-11-7f3a9c
```

The folder-like layout

S3 object layout - representative

```
s3://orders-etl-pipeline-2026-11-7f3a9c/
|
|-- raw/orders/
|   |-- orders_2026-11-10.csv
|   |-- orders_2026-11-11.csv
|
|-- curated/orders/
|   |-- order_date=2026-11-10/
|   |   |-- <generated-file>.snappy.parquet
|   |-- order_date=2026-11-11/
|       |-- <generated-file>.snappy.parquet
|
|-- quarantine/orders/
|   |-- orders_2026-11-10.csv
|
|-- reports/
|   |-- orders_2026-11-10.json
|   |-- orders_2026-11-11.json
|
|-- athena-results/
```

S3 uses object keys, not traditional folders. The slash-separated names make the objects look like folders in the console.

Security settings verified

- S3 Block Public Access settings were enabled.
- Default encryption was SSE-S3 (AES256). This is not the same as claiming customer-managed KMS encryption.
- The bucket was not made public.
- The Lambda's custom S3 policy was scoped to the project's required prefixes.

The raw, curated, quarantine and report prefixes have different jobs. Keeping them separate helps prevent accidental use of unvalidated data.

8. IAM and the automatic trigger

IAM: the worker's permission card

Lambda needs permission to read the incoming CSV, write output files and create CloudWatch logs. We created an execution role named `orders-etl-lambda-ingest-role`, attached the AWS managed basic execution policy for logs, and attached a custom S3 policy.

The custom policy allows reading objects under `raw/orders/`, writing to the curated, quarantine and reports prefixes, deleting curated objects for partition replacement, listing the relevant prefixes and reading the bucket location.

Why is delete permission present? The Parquet writer uses partition overwrite mode. Replacing a partition can require deleting prior objects in that partition. This permission is scoped to the curated prefix.

How S3 wakes up Lambda

Verified trigger and Lambda settings

```
Event source:
  S3 bucket: orders-etl-pipeline-2026-11-7f3a9c
  Event:     s3:ObjectCreated:*
  Prefix:    raw/orders/
  Suffix:    .csv

Lambda:
  orders-etl-ingest
  Runtime: Python 3.14
  Handler: lambda_function.handler
  Memory:  1024 MB
  Timeout: 120 seconds
```

The Lambda resource policy also grants the S3 service permission to invoke this function, scoped to the source bucket. The bucket notification configuration was read back and verified after creation.

What the trigger test proved

A CSV for 11 November was uploaded after the trigger was configured. The S3 quality report showed a processing timestamp of 19:55:39 UTC. The report object was listed at 19:55:40 UTC. The curated Parquet file for that date also appeared in S3. This is strong evidence of automatic S3-triggered processing; it was not a manual Lambda invocation.

9. Lambda 1: the data-quality worker

The first Lambda is named `orders-etl-ingest`. It is the heart of the part already built. Its job is to turn an untrusted CSV into clean analytical data plus a record of what was rejected.

Its work, in simple steps

- Read every S3 record in the event, not only the first one.
- Ignore objects that are not CSV files under the expected raw prefix.
- Read the CSV from S3.
- Check that required columns exist and that the file is not empty.
- Clean text and convert numeric/date columns to suitable types.
- Build validation checks and rejection reasons for each row.
- Keep the first row for an order ID and reject later duplicates within that file.
- Write valid rows as partitioned Parquet.
- Write rejected rows as a quarantine CSV.
- Write a JSON quality report.
- If the rejection rate is above the configured threshold and an SNS topic is configured, attempt to publish an alert.

Simplified validation example

Illustrative Python excerpt - not a replacement for the full Lambda

```
# Teaching example: simplified validation logic
invalid_quantity = (
    df["quantity"].isna()
    | (df["quantity"] <= 0)
    | (df["quantity"] % 1 != 0)
)

invalid_price = (
    df["unit_price"].isna()
    | (df["unit_price"] <= 0)
)

missing_customer = df["customer_id"].isna()
missing_timestamp = df["order_ts"].isna()
```

The real project code also checks required columns, parses timestamps, records row-level reasons, handles duplicates and writes the output objects. This excerpt only illustrates the rule style.

How the rejection rate is calculated

Formula and measured example

```
reject_rate = rejected_rows / total_rows

# Example:
250 / 5050 = 0.049505... = 4.95%
```

The configured threshold is 10%. The two normal test files had 4.95% rejected rows, so they should not trigger the threshold-based alert. SNS has not yet been configured, so no email alert has been verified.

10. Why Parquet? Why partition by date?

CSV is easy for humans to open, but it stores values as text separated by commas. Parquet is a columnar file format designed for analytical workloads. It stores column data efficiently and works well with Athena.

CSV	Parquet
Easy to inspect in a text editor or spreadsheet.	Designed for analytical tools.
Often repeats column names and text formatting.	Stores typed, column-oriented data.
Can require reading lots of text to analyse.	Athena can read only the columns needed for many queries.
Raw input format in this project.	Clean output format in this project.

Partitioning

Partitioning means placing data in separate paths based on a value. Here the value is `order_date`. Instead of one huge folder, each date has its own location:

Date partitions

```
curated/orders/order_date=2026-11-10/
curated/orders/order_date=2026-11-11/
```

When a query filters on `order_date`, Athena can use the partition metadata to avoid scanning unrelated date partitions. The actual amount scanned still depends on the query and data layout.

What the writer is configured to do

Representative awswrangler write configuration

```
wr.s3.to_parquet(
    df=valid,
    path="s3://orders-etl-pipeline-2026-11-7f3a9c/curated/orders/",
    dataset=True,
    mode="overwrite_partitions",
    partition_cols=["order_date"],
    compression="snappy",
    index=False
)
```

This is a representative excerpt of the configured Parquet write. The actual Lambda code should be treated as the source of truth. Partition overwrite is intended to replace the matching date partition; reprocessing and concurrent-write behaviour still need dedicated tests.

11. Glue Data Catalog: the library catalogue

A warehouse can contain thousands of boxes. A catalogue tells us what each box contains and where to find it. Glue Data Catalog plays that role for the Parquet data.

We created database `orders_analytics_db` and table `orders`. The table describes the columns and says that `order_date` is the partition key. The table points to the curated S3 prefix.

The registered table shape

Glue table schema - simplified

```
Database: orders_analytics_db
Table:    orders

Columns:
  order_id          string
  customer_id       bigint
  category          string
  quantity          bigint
  unit_price        double
  order_ts          timestamp
  source_row_number bigint

Partition:
  order_date        string

Format: Parquet
```

How we registered the second date

After the 11 November Parquet file was created, we ran Athena's partition discovery command:

Athena partition discovery

```
MSCK REPAIR TABLE orders_analytics_db.orders;
```

The query execution returned `SUCCEEDED`. We then called Glue `get-partition` for `2026-11-11` and verified that the partition pointed to the expected S3 path.

12. Athena: ask questions using SQL

Athena lets us query data in S3 with SQL. It uses the Glue table definition to understand the Parquet files. We created the workgroup `orders-etl-workgroup`, enforced its result location in S3 and configured SSE-S3 encryption for query results.

Daily KPI query

SQL used for the November 11 KPI query

```
SELECT
    order_date,
    COUNT(DISTINCT order_id) AS total_orders,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(quantity) AS units_sold,
    ROUND(SUM(quantity * unit_price), 2) AS total_revenue,
    ROUND(
        SUM(quantity * unit_price)
        / NULLIF(COUNT(DISTINCT order_id), 0),
        2
    ) AS average_order_value
FROM orders_analytics_db.orders
WHERE order_date = '2026-11-11'
GROUP BY order_date;
```

`NULLIF(..., 0)` protects the average calculation from division by zero. `COUNT(DISTINCT ...)` counts unique IDs, while `SUM(quantity * unit_price)` calculates revenue from the order lines.

13. Results: what the pipeline actually produced

These are measured results from the two synthetic daily datasets. Revenue and average order value are reported in the dataset's generated price units; no currency was specified in the verified data.

Data-quality results

Date	Input rows	Valid rows	Rejected rows	Reject rate
2026-11-10	5,050	4,800	250	4.95%
2026-11-11	5,050	4,800	250	4.95%

Rejection reasons - each date

Reason	Rejected rows
Missing customer ID	50
Invalid quantity	50
Invalid unit price	50
Missing order timestamp	50
Duplicate order ID	50
Missing order ID	0
Missing category	0
Total rejected rows	250

The reason counts add up to 250 for each file. The current validation report records the reasons assigned to rejected rows.

Athena KPI results

Metric	2026-11-10	2026-11-11
Total orders	4,800	4,800
Unique customers	3,589	3,606
Units sold	12,084	11,993
Total revenue	1,839,616.39	1,868,497.74
Average order value	383.25	389.27

How we checked the numbers

For 10 November, the Parquet file was downloaded and analysed locally. Its row count, unique orders, revenue and full KPI set matched Athena. For 11 November, Athena returned the KPI values above and the Parquet object was confirmed in S3. The separate local download and metric comparison for 11 November was still pending at the time this report was prepared.

Do not overstate: the 11 November Athena query succeeded, but its independent local Parquet comparison had not yet been completed.

14. Evidence and test trail

November 10 - manual Lambda test

A test S3 event JSON was used to invoke `orders-etl-ingest` synchronously. The response reported one processed file, 5,050 total rows, 4,800 valid rows, 250 rejected rows and a 4.9505% rejection rate. S3 output objects were then listed and the Parquet file was downloaded for local verification.

November 11 - automatic S3 event test

The bucket notification was configured for new CSV objects under `raw/orders/`. The 11 November file was uploaded after configuration. The quality report appeared in S3 with a processing timestamp of `2026-11-11T19:55:39.657441+00:00`; the object listing showed the report modified at `19:55:40+00:00`. The Parquet file was also present under the date partition.

Athena execution IDs

Purpose	Execution ID	Result
Nov 10 validation query	29babd2a-5fa2-4636-8a2e-fddee3772749	SUCCEEDED
Nov 10 daily KPI query	5b3fa6b6-36e4-4ca6-9bdc-640a6c83dbd5	SUCCEEDED
Nov 11 partition repair	b1942af4-6a78-4359-8d36-10601d4f9b3b	SUCCEEDED
Nov 11 daily KPI query	1ef6c9fe-4631-42b4-9328-1c1fa851b07d	SUCCEEDED

Execution IDs are useful evidence: an interviewer or reviewer can understand that the SQL was submitted to Athena and completed. The query results and local comparisons provide stronger correctness evidence than a successful status alone.

15. Useful commands to understand the build

Check the AWS identity and region

AWS CLI

```
aws sts get-caller-identity --profile etl-project
aws configure get region --profile etl-project
```

List the report generated by Lambda

PowerShell

```
aws s3api list-objects-v2 `
  --bucket orders-etl-pipeline-2026-11-7f3a9c `
  --prefix reports/orders_2026-11-11.json `
  --query "Contents[].{Key:Key,Size:Size,LastModified:LastModified}" `
  --output table `
  --profile etl-project `
  --region ap-southeast-2
```

Read the quality report from S3

PowerShell

```
aws s3 cp `
  s3://orders-etl-pipeline-2026-11-7f3a9c/reports/orders_2026-11-11.json `
  - `
  --profile etl-project `
  --region ap-southeast-2
```

Check a Glue partition

PowerShell

```
aws glue get-partition `
  --database-name orders_analytics_db `
  --table-name orders `
  --partition-values 2026-11-11 `
  --query "Partition.{Values:Values,Location:StorageDescriptor.Location}" `
  --output json `
  --profile etl-project `
  --region ap-southeast-2
```

PowerShell tip: copy only the command. Do not paste the terminal prompt (`(.venv) PS C:\...>`) or the previous command output. The `>>` characters are PowerShell continuation prompts, not part of the command.

16. What is still left to build?

The project is not yet the full AI-insights system described in the original idea. The next phases below are required to complete that larger scope.

A. SNS email alerts

- Create an SNS topic for ETL alerts.
- Subscribe an email address and confirm the subscription.
- Give the ingestion Lambda permission to publish to that topic.
- Configure the topic ARN in the Lambda environment.
- Test a normal file (no alert expected) and a high-defect file (alert expected).
- Verify that the email actually arrives; a successful publish response alone is not proof of delivery.

B. Lambda 2 + Gemini AI insights

A second Lambda, `orders-etl-insights`, is planned. It should query Athena for real daily KPIs, send those numbers to the Gemini API, receive a short explanation, save a JSON report in S3 and publish an email through SNS.

Illustrative pseudocode only

```
# Conceptual flow - not implemented code
date = event["date"]
kpis = run_athena_query(date)
summary = ask_gemini_to_explain(kpis)
report = {
    "date": date,
    "kpis": kpis,
    "ai_summary": summary
}
save_report_to_s3(report)
publish_report_email(report)
```

Python/SQL must calculate the metrics. Gemini should explain only the supplied values. It must not invent orders, revenue, causes or trends. The API key must be stored securely and must never be committed to Git or printed in logs.

C. EventBridge daily schedule

After Lambda 2 works manually, EventBridge Scheduler can run it once per day. The timezone should be `Asia/Kolkata`. The user still needs to choose the report time. The schedule must run after the daily ingestion is expected to finish.

D. Optional SQS + DLQ

SQS can buffer S3 events before Lambda processes them. A Dead Letter Queue stores messages that keep failing. This is optional reliability hardening. It changes the event path and needs careful configuration and testing; it should not be added until the direct S3-to-Lambda path is stable and the user approves it.

17. Security, reliability and known limitations

Security practices already in place

- S3 Block Public Access is enabled.
- S3 default encryption is SSE-S3 (AES256).
- The ingestion Lambda has a dedicated IAM execution role.
- The custom S3 policy is scoped to this project's bucket paths.
- The CLI uses browser-based login rather than permanent IAM user keys.
- The AWS account has a $5 monthly budget alert with 80% and 100% actual-spend notifications configured.

Things that still need review or testing

- Actual AWS billing was not checked for this report. A budget alert is a notification, not a hard spending cap.
- The current duplicate check is per file, not across all historical files.
- S3 notifications and Lambda processing can be delivered more than once; the pipeline must tolerate retries and duplicate events.
- Partition overwrite behaviour needs an isolated reprocessing test, including checking for stale or duplicate Parquet objects.
- Empty, malformed and schema-invalid CSV cases need automated test coverage.
- SNS email delivery, Gemini API failure handling and EventBridge execution are not yet verified.
- The Lambda execution role and any future policies should be reviewed for least privilege.

Never put AWS credentials, Gemini API keys or other secrets in source code, screenshots, Git commits or the final report. Redact account-specific or personal information before sharing the project publicly.

18. How to explain this project in an interview

30-second explanation

I built the foundation of a serverless AWS ETL pipeline for daily order data. A CSV is uploaded to S3, which automatically triggers a Python Lambda function. The function validates the records, separates invalid rows into a quarantine file, writes valid records as date-partitioned Parquet, and generates a JSON quality report. I registered the data in AWS Glue and queried it with Athena to calculate daily orders, customers, units sold, revenue and average order value. I verified the ingestion flow on two dates and matched the 10 November Athena results against the local Parquet data. The SNS alerting and Gemini-based insights stages are the next implementation phases.

Two-minute explanation

The problem I wanted to solve was that raw daily CSV files are not immediately reliable for analysis. They can contain missing fields, invalid quantities or prices, and duplicate order IDs. I created a generator to produce realistic synthetic order files with controlled data defects.

I stored the raw files in a private S3 bucket. An S3 object-created event invokes the ingestion Lambda. The Lambda checks the required schema, converts fields to appropriate types, records rejection reasons and separates valid from invalid rows. Valid data is stored as Snappy-compressed Parquet partitioned by order date. Rejected rows go to a quarantine prefix, and a JSON report records the input, valid and rejected counts.

I created an IAM role with scoped S3 permissions, a Glue database and table, and an Athena workgroup. I registered the date partitions and wrote SQL to calculate daily KPIs. For 10 November, I independently calculated the metrics from the downloaded Parquet file and matched the Athena output. I then uploaded a second date and verified automatic S3-triggered processing and the Athena query. The next stage is to add SNS email alerts, a second Lambda that uses Gemini to explain the verified KPIs, and a daily EventBridge schedule.

Technical deep-dive points

- Why Lambda? It runs code in response to events without managing a long-running server. The current function uses Python 3.14, 1,024 MB memory and a 120-second timeout.
- Why Parquet? It is columnar and suited to analytical queries; it also preserves useful data types.
- Why partition by date? It organises daily data and allows date-filtered queries to use partition metadata.
- Why Glue? Athena needs table metadata describing the schema, format and S3 location.
- Why Athena? It runs SQL directly over data in S3 without loading it into a database server.
- How did you validate correctness? I compared row counts and KPI calculations from the Parquet output with Athena query results.
- What is a limitation? Duplicate detection currently operates within one file; cross-file deduplication is not implemented.

19. Interview questions you should be ready for

What is ETL?

Extract, Transform, Load: collect raw data, clean/transform it, then store it for analysis.

What triggers Lambda?

An S3 ObjectCreated notification filtered to CSV objects under `raw/orders/`.

What happens to bad rows?

They are written to a quarantine CSV, and the quality report records rejection counts and reasons.

How is rejection rate calculated?

Rejected row count divided by total input row count.

Why is the threshold 10%?

It is the configured alert threshold. The two normal test files had 4.95%, below that threshold.

What is a Glue partition?

Metadata identifying a slice of a table, here one `order_date`, and its S3 location.

What does MSCK REPAIR TABLE do?

It discovers Hive-style partition directories and adds missing partition metadata to the catalog.

Why use `COUNT(DISTINCT order_id)`?

To count unique order IDs rather than simply counting rows.

How do you calculate revenue?

Sum of quantity multiplied by unit price for each valid order line.

Does the current pipeline deduplicate across dates?

No. Duplicate order IDs are detected within an individual file only.

Is the AI summary implemented?

Not yet. Gemini integration is a planned phase; no AI-generated report should be claimed as delivered.

How do you protect secrets?

Do not hardcode them; configure them securely outside source control and avoid logging their values.

Is serverless free?

No. Managed services can still incur usage charges. A budget alert is configured, but actual spend must be checked separately.

What would you improve next?

SNS alerts, robust automated tests, idempotent/retry-safe processing, Lambda 2 with Gemini, EventBridge scheduling and optional SQS/DLQ.

20. Small dictionary of important words

Word	Simple meaning
AWS	Amazon's cloud platform.
Bucket	A named storage container in S3.
Object / key	A file stored in S3 and its full name/path.
Lambda	A managed service that runs code when invoked.
Event	A message that says something happened, such as a new file upload.
IAM role	An identity that a service can assume to get permissions.
Policy	A document that says what actions are allowed or denied.
CSV	A text file where columns are separated by commas.
Parquet	A columnar file format used for analytics.
Partition	A logical slice of a dataset, such as one day.
Glue Data Catalog	Metadata catalogue for tables and partitions.
Athena	A serverless SQL query service for data in S3.
SNS	A publish/subscribe notification service.
EventBridge	A service for event routing and schedules.
SQS	A managed message queue.
DLQ	A dead-letter queue for messages that fail repeatedly.
KPI	Key Performance Indicator: a number used to measure performance.
AOV	Average Order Value: revenue divided by the number of orders.
Idempotency	Running the same operation again without creating an incorrect extra effect.
RAG	Retrieval-Augmented Generation; not part of the current ETL implementation.

21. Completion checklist and next actions

Use this list to continue the project without losing track of what is real and what is still planned.

Task	Current state
Download Nov 11 Parquet and compare locally with Athena	Pending at report creation
Create multi-day KPI SQL and verify both dates	Recommended next validation
Automated tests for valid/invalid/empty/malformed CSV	Pending
Reprocessing and idempotency test	Pending
Create SNS topic and confirm email subscription	Pending
Test high-defect and normal-day alert behaviour	Pending
Build and deploy Lambda 2	Pending
Configure Gemini API key securely and test real API call	Pending
Save AI insights JSON in S3 and verify	Pending
Send actual insights email through SNS	Pending
Choose daily report time and configure EventBridge	Pending
Optional SQS + DLQ design and test	Deferred / approval required
Finish README, requirements, .gitignore and diagrams	Pending
Check AWS billing and document actual spend	Pending
Create final project completion report with evidence	Pending

What is needed from the project owner

- An email address for the SNS subscription, followed by confirmation from the SNS email.
- A Gemini API key configured securely on the local machine or in an approved secret store.
- The preferred daily report time in Asia/Kolkata.
- Approval before optional SQS/DLQ resources are created.
- Explicit approval before pushing the project to GitHub.

Final honest summary

The verified foundation is real: the project has a working S3-to-Lambda ingestion path, validation and quarantine outputs, partitioned Parquet data, Glue metadata and successful Athena KPI queries for two dates. The complete AI-insights workflow is not finished yet. Completing SNS, Gemini, Lambda 2 and EventBridge, then testing and documenting them, will turn this foundation into the full project originally planned.

End of guide.